

Critical Thinking Module 4: Operating System First-Fit Memory Allocation

Alexander Ricciardi

Colorado State University Global

CSC507: Foundations of OSs

Professor: Dr. Joseph Issa

Jun 7, 2026

Critical Thinking Module 4: Operating System First-Fit Memory Allocation

Operating Systems (OSs) use a memory allocator to manage the Random Access Memory (RAM) in computer systems. The memory allocator's main job is to track where processes are loaded into RAM. The main challenge is to manage processes that have different sizes and need to be loaded into RAM over time. Moreover, an OS memory allocator's duties involve deciding how to track free memory space, protect active processes, and keep memory usable instead of letting it break into scattered fragments. Additionally, the memory allocator must keep track of addresses, coordinate access, and handle memory fragmentation, which makes the memory total free space look sufficient even though there is no single contiguous block that is large enough to meet the memory needs of a large process (CSU Global, n.d.).

This paper examines the First-Fit algorithm using a Python program. The Python script code logic simulates a dynamic memory allocation system that an OS may use. The program also implements different memory block sizes, process sizes, and request patterns to test the simulated memory allocation system. To match the First-Fit algorithm functionality, the program stores free memory as a list of contiguous holes, allocates each process to the first hole large enough to hold it, and records the outcome of each decision; see Figure 1 and Figure 2. Additionally, the program runs three scenarios to test first-fit, best-fit, and side-by-side comparison functionality, exporting to outputs to CSV files; see Figure 3. This allows the outputs to be examined. The examination of the outputs showed that First-Fit algorithm is more efficient and practical as it is simpler and inspects fewer holes. On the other hand, the Best-Fit algorithm is more suited to situations where memory is restricted and preserving larger holes for later searches matters more than search cost.

The First-Fit Algorithm

The First-Fit algorithm is a sequential memory allocation approach that minimizes overhead created by memory search; it implements a simple rule that assigns a first-come process to the very first available block of free memory that is large enough to accommodate it (Wilson et al., 1995). First, it scans a free-hole list from the lowest address to the highest address and stops at the first large enough hole to fit the process memory requirement. If the hole is larger than the process, the OS places the process at the front of that hole and leaves the remainder available as a smaller free block. If no hole is large enough, the allocation fails. This allocation implementation is simple, but it does not solve the larger fragmentation problem created by dynamic partitioning. This flexibility provided by the dynamic approach is valuable, but it also partitions memory free space into pieces that are too small to accommodate large processes' memory needs, even when the total free memory is sufficient to accommodate these processes' memory needs. This means that the memory allocator efficiency needs to be judged not only by how quickly it answers one request, but also by the shape of the memory that it leaves free for the next request. In other words, the First-Fit algorithm favors low search overhead over fully scanning the free hole list. The program associated with the project implements that exact functionality, see Figure 2. For each process, it walks the current free hole list, records the placement address when a big enough hole is found, subtracts the process size from that hole, and keeps any remainder in the free list. When no large enough hole is found, it records a failed allocation and identifies whether the failure was caused by external fragmentation rather than by an absolute lack of free memory.

Figure 1

Program setup

```

first_fit_memory_allocation.py U X
CTAs > CAT-4 > first_fit_memory_allocation.py > ...
1 # File: first_fit_memory_allocation.py
2 # Author: Alexander Ricciardi
3 # Date: 2026-06-07
4 # Course: CSC507
5 # Professor: Dr. Joseph Issa
6 # Term: Spring C 2026
7
8 """The Program simulates contiguous memory allocation with the First-Fit Algorithm.
9
10 This standalone Python script simulates free memory as an ordered list of holes.
11 It simulates three distinct memory allocation scenarios or a custom set of memory
12 blocks and process sizes. The script can run in one of three modes:
13 compare, first-fit, or best-fit. First-Fit and Best-Fit can be compared side by side
14 and summary metrics can be exported to CSV.
15 """
16
17 from __future__ import annotations
18
19 import argparse
20 import csv
21 from dataclasses import dataclass
22 from pathlib import Path
23
24 # =====
25 # Constants
26 # =====
27
28 DEFAULT_CSV_PATH = Path("first_fit_results.csv")
29 VALID_MODES = ("compare", "first-fit", "best-fit")
30
31 # =====
32 # Data Models / Classes
33 # =====
34
35 # ---- class MemoryHole
36 @dataclass(frozen=True)
37 class MemoryHole:
38     """Represent one free contiguous memory region.
39
40     Args:
41         start: Starting address of the free hole.
42         size: Number of addressable units in the free hole.
43
44     Returns:
45         None.
46     """
47
48     start: int
49     size: int
50
51 # ---- end class MemoryHole
52
53 # ---- class AllocationEvent
54 @dataclass(frozen=True)
55 class AllocationEvent:
56     """Store the result of one process-allocation attempt.
57
58     Args:
59         process_label: Process label such as P1 or P2.
60         process_size: Requested memory size.
61         allocated: Whether the process was placed into memory.
62         block_start: Starting address of the selected hole.
63         block_size_before: Size of the selected hole before placement.
64         remainder_size: Remaining free size after placement.
65         holes_inspected: Number of free holes inspected for this request.
66         fragmentation_blocked: Whether failure occurred despite enough total
67         free memory being available across smaller holes.
68
69     Returns:
70         None.
71     """
72
73     process_label: str # Label for the process
74     process_size: int # Size of the process
75     allocated: bool # Whether the process was allocated
76     block_start: int | None # Starting address of the selected hole
77     block_size_before: int | None # Size of the selected hole before placement
78     remainder_size: int | None # Remaining free size after placement
79     holes_inspected: int # Number of free holes inspected for this request
80     fragmentation_blocked: bool # Whether failure occurred despite enough total
81
82 # ---- end class AllocationEvent
83
84 # ---- class SimulationSummary
85 @dataclass(frozen=True)
86 class SimulationSummary:
87     """Store one finished allocation simulation.
88
89     Args:
90         scenario_name: Human-readable scenario name.
91         strategy_name: Allocation strategy name.
92         block_sizes: Starting free-memory block sizes.
93         process_sizes: Requested process sizes.
94         events: Ordered allocation events for all processes.
95         remaining_holes: Free holes remaining after the simulation.
96         total_free_memory: Sum of all remaining free memory.
97         largest_hole: Size of the largest remaining hole.
98         total_hole_inspections: Total free-hole checks performed.
99         fragmentation_failures: Count of failures caused by fragmentation.
100
101     Returns:
102         None.
103     """
104
105     scenario_name: str # Human-readable scenario name.
106     strategy_name: str # Allocation strategy name.
107     block_sizes: list[int] # Starting free-memory block sizes.
108     process_sizes: list[int] # Requested process sizes.
109     events: list[AllocationEvent] # Ordered allocation events for all processes.
110     remaining_holes: list[MemoryHole] # Free holes remaining after the simulation.
111     total_free_memory: int # Sum of all remaining free memory.
112     largest_hole: int # Size of the largest remaining hole.
113     total_hole_inspections: int # Total free-hole checks performed.
114     fragmentation_failures: int # Count of failures caused by fragmentation.
115
116 # ---- allocated_count()
117 def allocated_count(self) -> int:
118     """Count successfully allocated processes.
119
120     Args:
121         None.
122
123     Returns:
124         Number of successful allocations.
125     """
126     return sum(1 for event in self.events if event.allocated)
127
128 # ---- end allocated_count()
129
130
131
132 # ---- failed_count()
133 def failed_count(self) -> int:
134     """Count failed process allocations.
135
136     Args:
137         None.
138
139     Returns:
140         Number of failed allocations.
141     """
142     return sum(1 for event in self.events if not event.allocated)
143
144 # ---- end failed_count()
145
146 # ---- to_csv_row()
147 def to_csv_row(self) -> dict[str, str | int]:
148     """Convert the summary to a CSV-ready dictionary.
149
150     Args:
151         None.
152
153     Returns:
154         Dictionary containing CSV columns.
155     """
156     return {
157         "scenario": self.scenario_name,
158         "strategy": self.strategy_name,
159         "allocated": self.allocated_count(),
160         "failed": self.failed_count(),
161         "fragmentation_failures": self.fragmentation_failures,
162         "total_free_memory": self.total_free_memory,
163         "largest_hole": self.largest_hole,
164         "remaining_hole_count": len(self.remaining_holes),
165         "total_hole_inspections": self.total_hole_inspections,
166         "block_sizes": ",".join(str(size) for size in self.block_sizes),
167         "process_sizes": ",".join(str(size) for size in self.process_sizes),
168         "remaining_hole_sizes": ",".join(
169             str(hole.size) for hole in self.remaining_holes
170         ),
171     }
172
173 # ---- end to_csv_row()
174
175 # ---- end class SimulationSummary
176
177 # ---- class ScenarioDefinition
178 @dataclass(frozen=True)
179 class ScenarioDefinition:
180     """Store one named scenario used by the simulator.
181
182     Args:
183         name: Human-readable scenario name.
184         block_sizes: Starting free-memory block sizes.
185         process_sizes: Requested process sizes.
186         note: Short explanation of why the scenario is useful.
187
188     Returns:
189         None.
190     """
191
192     name: str # Human-readable scenario name.
193     block_sizes: list[int] # Starting free-memory block sizes.
194     process_sizes: list[int] # Requested process sizes.
195     note: str # Short explanation of why the scenario is useful.
196
197 # ---- end class ScenarioDefinition
198
199 # =====
200 # Scenario Catalog
201 # =====
202
203 # ---- Dictionary of built-in scenarios
204 BUILT_IN_SCENARIOS: dict[str, ScenarioDefinition] = {
205     "balanced": ScenarioDefinition(
206         name="Balanced mixed-size requests", # Name of the scenario
207         block_sizes=[100, 500, 200, 300, 600], # Free memory block sizes
208         process_sizes=[212, 417, 112, 426], # Process sizes to be allocated
209         note=(
210             "Shows how a later large request can fail under First-Fit even when "
211             "the system still has enough total free memory."
212         ),
213     ),
214     "clustered": ScenarioDefinition(
215         name="Clustered small and medium requests", # Name of the scenario
216         block_sizes=[150, 350, 120, 500, 275], # Free memory block sizes
217         process_sizes=[115, 90, 130, 260, 50, 200], # Process sizes to be allocated
218         note=(
219             "Shows a workload where both strategies allocate every process, but "
220             "Best-Fit spends more search effort and leaves very small fragments."
221         ),
222     ),
223     "pressure": ScenarioDefinition(
224         name="High fragmentation pressure", # Name of the scenario
225         block_sizes=[400, 180, 220, 260, 300], # Free memory block sizes
226         process_sizes=[175, 205, 90, 240, 60, 180, 110], # Process sizes to be allocated
227         note=(
228             "Pushes several medium and small requests through the allocator to "
229             "compare final hole layout under sustained pressure."
230         ),
231     ),
232 }
233
234 # ---- End dictionary of built-in scenarios
235
236 # =====
237 # Parsing and Validation Helpers
238 # =====
239
240 # ---- parse_positive_csv()
241 def parse_positive_csv(raw_value: str) -> list[int]:
242     """Parse a comma-separated list of positive integers.
243
244     Args:
245         raw_value: Raw comma-separated string.
246
247     Returns:
248         Parsed list of positive integers.
249
250     Raises:
251         ValueError: If the input is empty or contains invalid numbers.
252     """
253     # Initialize an empty list to store parsed values
254     parsed_values: list[int] = []
255
256     # Split the raw value by commas and iterate through each piece
257     for piece in raw_value.split(","):
258         # Remove leading/trailing whitespace from each piece
259         cleaned_piece = piece.strip()
260         # Skip empty pieces
261         if not cleaned_piece:
262             continue
263
264         # Convert the cleaned piece to an integer
265         parsed_value = int(cleaned_piece)
266
267         # Check if the parsed value is positive
268         if parsed_value <= 0:
269             raise ValueError("at least one positive integer is required")
270
271         # Append the parsed value to the list
272         parsed_values.append(parsed_value)
273
274     # Check if the list is empty
275     if not parsed_values:
276         raise ValueError("at least one positive integer is required")
277
278     # Return the list of parsed values
279     return parsed_values
280
281 # ---- end parse_positive_csv()
282
283 # ---- build_memory_holes()
284 def build_memory_holes(block_sizes: list[int]) -> list[MemoryHole]:
285     """Build the starting free-hole list from block sizes.
286
287     Args:
288         block_sizes: Sizes of the starting free holes.
289
290     Returns:
291         Ordered list of free memory holes.
292     """
293     # Initialize an empty list to store free holes
294     holes: list[MemoryHole] = []
295     # Initialize the starting address of the next hole
296     next_start = 0
297
298     # Iterate through the block sizes
299     for block_size in block_sizes:
300         # Create a new hole with the current block size
301         holes.append(MemoryHole(start=next_start, size=block_size))
302         # Update the starting address for the next hole
303         next_start += block_size
304
305     return holes
306
307 # ---- end build_memory_holes()

```

Note: The image shows Python code used to set up the program; the code uses data classes to store memory holes, allocation events, simulation summaries, and scenario definitions.

Figure 2

Python Code First-Fit Core Logic

```

311 # ----- select_first_fit_index()
312 # The code below scans the free-hole list from left to right and selects the
313 # first memory hole large enough to hold the incoming process. This is the core
314 # First-Fit rule used by the memory allocation simulator.
315 # =====
316
317 # --- select_first_fit_index()
318 def select_first_fit_index(
319     holes: list[MemoryHole],
320     process_size: int,
321 ) -> tuple[int | None, int]:
322     """Select the first adequate hole.
323
324     Args:
325         holes: Ordered list of free holes.
326         process_size: Size of the process being allocated.
327
328     Returns:
329         Tuple containing the selected index and the number of inspected holes.
330     """
331     # Initialize the number of inspected holes
332     inspected_holes = 0
333     # Iterate through the free holes
334     for hole_index, hole in enumerate(holes):
335         # Increment the number of inspected holes
336         inspected_holes += 1
337         # Check if the current hole is large enough for the process
338         if hole.size >= process_size:
339             # Return the index of the selected hole and the number of inspected holes
340             return hole_index, inspected_holes
341     # Return None and the number of inspected holes if no adequate hole is found
342     return None, inspected_holes
343
344 # --- end select_first_fit_index()
345
346 # --- select_best_fit_index()
347 def select_best_fit_index(
348     holes: list[MemoryHole], # Ordered list of free holes
349     process_size: int, # Size of the process being allocated
350 ) -> tuple[int | None, int]:
351     """Select the smallest adequate hole.
352
353     Args:
354         holes: Ordered list of free holes.
355         process_size: Size of the process being allocated.
356
357     Returns:
358         Tuple containing the selected index and the number of inspected holes.
359     """
360     # Initialize the number of inspected holes
361     inspected_holes = 0
362     # Initialize the index of the best fit hole
363     best_index: int | None = None
364     # Initialize the size of the best fit hole
365     best_size: int | None = None
366
367     # Iterate through the free holes
368     for hole_index, hole in enumerate(holes):
369         # Increment the number of inspected holes
370         inspected_holes += 1
371         # Skip the hole if it is smaller than the process size
372         if hole.size < process_size:
373             continue
374         # Update the best fit hole if the current hole is smaller than the best fit hole
375         if best_size is None or hole.size < best_size:
376             # Update the best fit index
377             best_index = hole_index
378             # Update the best fit size
379             best_size = hole.size
380
381     return best_index, inspected_holes
382
383 # --- end select_best_fit_index()
384
385 # --- simulate_allocation()
386 def simulate_allocation(
387     scenario_name: str, # Human-readable scenario name
388     block_sizes: list[int], # Starting free-memory block sizes
389     process_sizes: list[int], # Requested process sizes
390     strategy_name: str, # "First-Fit" or "Best-Fit"
391 ) -> SimulationSummary: # Completed simulation summary
392     """Run one full allocation simulation.
393
394     Args:
395         scenario_name: Human-readable scenario name.
396         block_sizes: Starting free-memory block sizes.
397         process_sizes: Requested process sizes.
398         strategy_name: Either "First-Fit" or "Best-Fit".
399
400     Returns:
401         Completed simulation summary.
402     """
403     # Build the initial list of free holes
404     holes = build_memory_holes(block_sizes)
405     # Initialize an empty list to store allocation events
406     events: list[AllocationEvent] = []
407     # Initialize the total number of hole inspections
408     total_hole_inspections = 0
409     # Initialize the number of fragmentation failures
410     fragmentation_failures = 0
411
412     # Select the appropriate selector based on the strategy name
413     selector = (
414         select_first_fit_index
415         if strategy_name == "First-Fit"
416         else select_best_fit_index
417     )
418
419     # Step 1: Attempt each allocation request in order.
420     for process_number, process_size in enumerate(process_sizes, start=1):
421         # Create a process label
422         process_label = f"P{process_number}"
423         # Select the appropriate hole for the process
424         selected_index, inspected_holes = selector(holes, process_size)
425         # Increment the total number of hole inspections
426         total_hole_inspections += inspected_holes
427
428         # VALIDATION: Detect fragmentation when total free memory exists but no
429         # single hole is large enough for the request.
430         if selected_index is None: # If no adequate hole is found
431             # Calculate the total free memory
432             total_free_memory = sum(hole.size for hole in holes)
433             # Determine if fragmentation is blocking the allocation
434             fragmentation_blocked = total_free_memory >= process_size
435             # Increment the number of fragmentation failures if fragmentation is blocking
436             if fragmentation_blocked:
437                 fragmentation_failures += 1
438
439         # Append the allocation event
440         events.append(
441             AllocationEvent(
442                 process_label=process_label, # Process label
443                 process_size=process_size, # Process size
444                 allocated=False, # Whether the process was allocated
445                 block_start=None, # Starting address of the block
446                 block_size_before=None, # Size of the block before allocation
447                 remainder_size=None, # Size of the remainder
448                 holes_inspected=inspected_holes, # Number of holes inspected
449                 fragmentation_blocked=fragmentation_blocked, # Whether fragmentation blocked the allocation
450             )
451         )
452         continue
453
454     # Get the selected hole
455     selected_hole = holes[selected_index]
456     # Calculate the remainder size
457     remainder_size = selected_hole.size - process_size
458
459     # Append the allocation event
460     events.append(
461         AllocationEvent(
462             process_label=process_label, # Process label
463             process_size=process_size, # Process size
464             allocated=True, # Whether the process was allocated
465             block_start=selected_hole.start, # Starting address of the block
466             block_size_before=selected_hole.size, # Size of the block before allocation
467             remainder_size=remainder_size, # Size of the remainder
468             holes_inspected=inspected_holes, # Number of holes inspected
469             fragmentation_blocked=False, # Whether fragmentation blocked the allocation
470         )
471     )
472
473     # Step 2: Shrink or remove the hole after the allocation.
474     if remainder_size == 0: # If the remainder is zero
475         holes.pop(selected_index) # Remove the hole
476     else: # If the remainder is not zero
477         holes[selected_index] = MemoryHole( # Update the hole
478             start=selected_hole.start + process_size, # Update the starting address
479             size=remainder_size, # Update the hole size
480         )
481
482     # Calculate the total free memory
483     total_free_memory = sum(hole.size for hole in holes)
484     # Find the largest hole
485     largest_hole = max(hole.size for hole in holes, default=0)
486
487     # Return the simulation summary
488     return SimulationSummary(
489         scenario_name=scenario_name, # Readable scenario name
490         strategy_name=strategy_name, # "First-Fit" or "Best-Fit"
491         block_sizes=list(block_sizes), # Starting free-memory block sizes
492         process_sizes=list(process_sizes), # Requested process sizes
493         events=events, # List of allocation events
494         remaining_holes=list(holes), # List of remaining free holes
495         total_free_memory=total_free_memory, # Total free memory
496         largest_hole=largest_hole, # Largest hole
497         total_hole_inspections=total_hole_inspections, # Total hole inspections
498         fragmentation_failures=fragmentation_failures, # Fragmentation failures
499     )
500
501 # --- end simulate_allocation()
502
503 # =====
504 # Reporting Helpers
505 # =====
506
507 # --- format_event_table()
508 def format_event_table(summary: SimulationSummary) -> str:
509     """Format a readable per-process allocation table.
510
511     Args:
512         summary: Completed simulation summary.
513
514     Returns:
515         Multi-line table string.
516     """
517     # Initialize the table with headers
518     lines = [
519         "Process Size Result Inspected",
520         "-----",
521     ]
522
523     # Iterate through the events
524     for event in summary.events:
525         # If the process was allocated
526         if event.allocated:
527             # Format the result text
528             result_text = (
529                 f"addr {event.block_start}, hole {event.block_size_before}, "
530                 f"left {event.remainder_size}"
531             )
532         # If the process was not allocated due to external fragmentation
533         elif event.fragmentation_blocked:
534             result_text = "not allocated (external fragmentation)"
535         # If the process was not allocated due to insufficient memory
536         else:
537             result_text = "not allocated (insufficient memory)"
538
539         # Append the event to the table
540         lines.append(
541             f"{event.process_label:<7} "
542             f"{event.process_size:>4} "
543             f"{result_text:<36} "
544             f"{event.holes_inspected:>9}"
545         )
546
547     return "\n".join(lines)
548
549 # --- end format_event_table()
550
551 # --- format_remaining_holes()
552 def format_remaining_holes(holes: list[MemoryHole]) -> str:
553     """Format the remaining holes for console output.
554
555     Args:
556         holes: Remaining free holes.
557
558     Returns:
559         Human-readable hole description.
560     """
561     # If there are no remaining holes, return "None"
562     if not holes:
563         return "None"
564
565     # Format the remaining holes for console output
566     return ", ".join(
567         f"(start={hole.start}, size={hole.size})"
568         for hole in holes
569     )
570
571 # --- end format_remaining_holes()
572
573 # --- format_comparison_table()
574 def format_comparison_table(summaries: list[SimulationSummary]) -> str:
575     """Format the scenario comparison summary table.
576
577     Args:
578         summaries: Collection of completed simulation summaries.
579
580     Returns:
581         Multi-line summary table.
582     """
583     # Initialize the table with headers
584     lines = [
585         "Scenario Strategy Alloc Fail Free Largest Inspections",
586         "-----",
587     ]
588
589     # Iterate through the summaries
590     for summary in summaries:
591         lines.append(
592             f"{summary.scenario_name[:38]:<38} "
593             f"{summary.strategy_name:<9} "
594             f"{summary.allocated_count():>5} "
595             f"{summary.failed_count():>4} "
596             f"{summary.total_free_memory:>4} "
597             f"{summary.largest_hole:>7} "
598             f"{summary.total_hole_inspections:>11}"
599         )
600
601     return "\n".join(lines)
602
603 # --- end format_comparison_table()
604
605 # --- write_csv_report()
606 def write_csv_report(
607     summaries: list[SimulationSummary],
608     output_path: Path,
609 ) -> None:
610     """Write summary rows to CSV.
611
612     Args:
613         summaries: Collection of simulation summaries.
614         output_path: Destination CSV path.
615
616     Returns:
617         None.
618     """
619     field_names = [
620         "scenario",
621         "strategy",
622         "allocated",
623         "failed",
624         "fragmentation_failures",
625         "total_free_memory",
626         "largest_hole",
627         "remaining_hole_count",
628         "total_hole_inspections",
629         "block_sizes",
630         "process_sizes",
631         "remaining_hole_sizes",
632     ]
633
634     # Open the CSV file
635     with output_path.open("w", encoding="utf-8", newline="") as csv_file:
636         writer = csv.DictWriter(csv_file, fieldnames=field_names)
637         writer.writeheader()
638         # Write the summaries to the CSV file
639         for summary in summaries:
640             writer.writerow(summary.to_csv_row())
641
642 # --- end write_csv_report()
643
644 # =====
645 # Command-Line Wiring
646 # =====
647
648 # --- parse_args()
649 def parse_args() -> argparse.Namespace:
650     """Parse command-line arguments.
651
652     Args:
653         None.
654
655     Returns:
656         Parsed argument namespace.
657     """
658     # Create the parser
659     parser = argparse.ArgumentParser(
660         description=(
661             "Simulate First-Fit memory allocation and optionally compare it "
662             "with Best-Fit."
663         )
664     )
665
666     # Add arguments
667     parser.add_argument(
668         "--scenario",
669         choices=(BUILT_IN_SCENARIOS.keys(), "all"),
670         default="all",
671         help="Built-in scenario to run. Default: all.",
672     )
673
674     # Add arguments
675     parser.add_argument(
676         "--mode",
677         choices=(VALID_MODES,
678             "compare",
679             "run only First-Fit, only Best-Fit, or both. Default: compare.",
680         ),
681     )
682
683     # Add arguments
684     parser.add_argument(
685         "--blocks",
686         help="Comma-separated custom free-block sizes.",
687     )
688
689     # Add arguments
690     parser.add_argument(
691         "--processes",
692         help="Comma-separated custom process sizes.",
693     )
694
695     # Add arguments
696     parser.add_argument(
697         "--csv",
698         type=Path,
699         help="Optional CSV output path for summary results.",
700     )
701
702     # Parse the arguments
703     return parser.parse_args()
704

```

Note: The image shows Python code used to scan the free-hole list and place each process under the First-Fit rule, it shows the core logic of First-Fit algorithm.

Figure 2

Python Code Test scenarios and First-Fit vs Best-Fit.

```
705 # =====
706 # SCENARIO TESTING AND FIRST-FIT / BEST-FIT COMPARISON
707 # =====
708 # The code below runs three built-in memory allocation scenarios or a custom set
709 # of memory blocks and process sizes. It supports compare, first-fit, and
710 # best-fit modes, allowing First-Fit and Best-Fit to be compared side by side.
711 # =====
712
713 # ---- run_selected_scenarios()
714 def run_selected_scenarios(arguments: argparse.Namespace) -> list[SimulationSummary]:
715     """Run the requested scenarios and strategies.
716
717     Args:
718         arguments: Parsed command-line arguments.
719
720     Returns:
721         Completed simulation summaries.
722
723     Raises:
724         ValueError: If custom blocks and processes are not both supplied.
725     """
726
727     # Check if custom blocks and processes are both supplied
728     if bool(arguments.blocks) != bool(arguments.processes):
729         raise ValueError(
730             "custom runs require both --blocks and --processes"
731         )
732
733     # Run custom scenarios if both blocks and processes are supplied
734     if arguments.blocks and arguments.processes:
735         block_sizes = parse_positive_csv(arguments.blocks)
736         process_sizes = parse_positive_csv(arguments.processes)
737         scenarios = [
738             ScenarioDefinition(
739                 name="Custom scenario",
740                 block_sizes=block_sizes,
741                 process_sizes=process_sizes,
742                 note="Custom command-line scenario.",
743             )
744         ]
745
746     # Run built-in scenarios if "all" is specified
747     elif arguments.scenario == "all":
748         scenarios = list(BUILT_IN_SCENARIOS.values())
749     # Run a single built-in scenario
750     else:
751         scenarios = [BUILT_IN_SCENARIOS[arguments.scenario]]
752
753     # Determine which strategies to run
754     strategy_names = (
755         ["First-Fit", "Best-Fit"]
756         if arguments.mode == "compare"
757         else [arguments.mode.title()]
758     )
759
760     summaries: list[SimulationSummary] = []
761
762     # Iterate through scenarios and strategies
763     for scenario in scenarios:
764         for strategy_name in strategy_names:
765             # Simulate allocation
766             summaries.append(
767                 simulate_allocation(
768                     scenario_name=scenario.name,
769                     block_sizes=scenario.block_sizes,
770                     process_sizes=scenario.process_sizes,
771                     strategy_name=strategy_name,
772                 )
773             )
774
775     return summaries
776
777 # ---- end run_selected_scenarios()
778
779 # ---- print_report()
780 def print_report(summaries: list[SimulationSummary]) -> None:
781     """Print the detailed scenario report.
782
783     Args:
784         summaries: Completed simulation summaries.
785
786     Returns:
787         None.
788     """
789     previous_scenario: str | None = None
790
791     # Iterate through summaries
792     for summary in summaries:
793         # Print scenario header if it's a new scenario
794         if summary.scenario_name != previous_scenario:
795             if previous_scenario is not None:
796                 print()
797             print(f"Scenario: {summary.scenario_name}")
798             print(f"Memory blocks: {summary.block_sizes}")
799             print(f"Process sizes: {summary.process_sizes}")
800
801         # Print strategy header
802         print()
803         print(f"Strategy: {summary.strategy_name}")
804
805         # Print event table
806         print(format_event_table(summary))
807
808         # Print summary
809         print(
810             "Summary: "
811             f"allocated {summary.allocated_count()}/{len(summary.process_sizes)}, "
812             f"failed {summary.failed_count()}, "
813             f"free memory {summary.total_free_memory}, "
814             f"largest hole {summary.largest_hole}, "
815             f"remaining holes {len(summary.remaining_holes)}, "
816             f"inspections {summary.total_hole_inspections}, "
817             f"fragmentation failures {summary.fragmentation_failures}"
818         )
819
820         # Print remaining holes
821         print(f"Remaining holes: {format_remaining_holes(summary.remaining_holes)}")
822
823         previous_scenario = summary.scenario_name
824
825     # Print comparison summary if more than one summary
826     if len(summaries) > 1:
827         print()
828         print("Comparison Summary")
829         print(format_comparison_table(summaries))
830
831 # ---- end print_report()
832
833 # ---- main()
834 def main() -> int:
835     """Run the command-line program.
836
837     Args:
838         None.
839
840     Returns:
841         Process exit code.
842     """
843     arguments = parse_args()
844     summaries = run_selected_scenarios(arguments)
845     print_report(summaries)
846
847     if arguments.csv:
848         write_csv_report(summaries, arguments.csv)
849         print()
850         print(f"CSV summary written to: {arguments.csv}")
851
852     return 0
853
854 # ---- end main()
855
856 if __name__ == "__main__":
857     raise SystemExit(main())
```

Note: The image shows the Python code used to run multiple test scenarios and compare First-Fit with Best-Fit.

Scenario Testing and Comparison with Best-Fit

Best-Fit uses a different decision rule. Instead of stopping at the first workable hole, it searches all available holes and chooses the smallest hole that can still satisfy the request. The goal is to reduce immediately wasted space inside the selected block and, when possible, preserve larger holes for future allocations. The tradeoff is that Best-Fit usually performs more search work because it must inspect every candidate hole before it can decide. This distinction matters because different request patterns stress the allocator in different ways. A mixed-size workload places more pressure on preserving contiguous space for later large requests, while a clustered small-and-medium workload places more emphasis on search overhead and leftover fragment quality. That pattern appeared clearly in the scenario tests. The simulator was run against three different memory layouts and process-request sequences: a balanced mixed-size case, a clustered small-and-medium case, and a high-fragmentation-pressure case. This testing approach fits the assignment requirement because each run changes the available memory block sizes, the process sizes, or both. See Table 1 for scenario outcomes.

Table 1

First-Fit VS Best-Fit Scenarios Outcomes

Scenario	Strategy	Allocated	Failed	Total Free	Largest Hole	Hole Inspections
Balanced mixed-size requests	First-Fit	3	1	959	300	14
Balanced mixed-size requests	Best-Fit	4	0	533	174	20
Clustered small and medium requests	First-Fit	6	0	550	275	15
Clustered small and medium requests	Best-Fit	6	0	550	300	30
High fragmentation pressure	First-Fit	7	0	300	190	18

High fragmentation pressure	Best-Fit	7	0	300	220	33
------------------------------------	----------	---	---	-----	-----	----

Note: The table shows First-Fit and Best-Fit across three memory-allocation scenarios' outputs produced by using the Python code. These measurements have been generated by the Program associated with this paper; see Figure 2 for the different scenarios' Python code.

The balanced scenario is the strongest example of why the algorithms can behave differently. With memory blocks `[100, 500, 200, 300, 600]` and process sizes `[212, 417, 112, 426]`, First-Fit allocated only three of the four processes. The final process failed even though 959 units of memory were still free in total. This was an external-fragmentation failure, because the free memory had been split across holes of sizes 100, 176, 200, 300, and 183, and none of those holes was large enough for the 426-unit request. Best-Fit allocated all four processes in the same scenario because it preserved the 600-unit hole for the later large request. The clustered and pressure scenarios reveal a different pattern. In both cases, First-Fit and Best-Fit allocated every process successfully, but Best-Fit inspected many more holes in order to decide where to place each request. In the clustered scenario, First-Fit completed the run with 15 hole inspections, while Best-Fit required 30. In the pressure scenario, First-Fit required 18 inspections and Best-Fit required 33. This supports the usual practical argument for First-Fit: it often makes an acceptable decision with less search overhead. The simulation also showed that Best-Fit can leave very small fragments behind. In the clustered scenario, Best-Fit left holes of 10 and 5 units, which are much less reusable than the larger leftover holes preserved by First-Fit. That matters because a memory-allocation policy is not judged only by one allocation decision. It is judged by the shape of the memory that remains after many decisions. Esmaili and Toghraee (2024) explain that concurrent systems depend on careful resource allocation and management across

many simultaneous activities. That broader systems perspective is useful here because memory-allocation quality affects what later work the system can still accept.

When Best-Fit Is Preferable

The Best-Fit algorithm is better suited when allocation success for later medium or large requests matters more than the extra search cost needed to find the smallest adequate hole. The balanced scenario from this project demonstrates that situation clearly. Best-Fit makes sure that a sufficiently large block is allocated for the final 426-unit process, while First-Fit uses the large block earlier and turns its remaining free space into smaller pieces. Best-Fit is also more suited to situations where memory is constrained but CPU time is not. For example, if memory allocation requests arrive at a moderate rate, then the CPU time cost of scanning more holes may be acceptable. This is especially true for server systems that host many differently sized processes over time, because fragmentation can accumulate gradually rather than happen at once.

Another situation where Best-Fit can be better suited is an analytical or batch processing environment where memory allocations and system stability are more important than latency. In other words, First-Fit is best for simple and fast decision making situations, and Best-Fit is best when the system can afford to execute full free hole list scans and the system benefits from preserving larger holes for future jobs. The logical conclusion that can be made is that Best-Fit is not universally better, but it is often better when memory is the main constraint. The clustered scenario showed that the Best-Fit algorithm can create tiny leftover fragments even when it allocates every process's memory needs successfully. Pandikumar et al. (2025) describe memory management or memory allocation as an optimization problem rather than a one-rule-fits-all problem. They argue that fragmentation management and adaptive memory methods for memory allocation need active improvement, especially in systems or workloads that try to balance

efficiency, flexibility, and allocation quality. In other words, the choice between using First-Fit algorithm or the Best-Fit algorithm is a matter of tradeoff and workload requirements.

Conclusion

As shown by the Python program outputs and the Best-Fit comparison, the First-Fit algorithm is easy to implement and effective for many ordinary workloads. The Python scripts illustrated the functionality of the algorithm by allocating each process into the first large enough memory hole, splitting the hole when necessary, and identifying external-fragmentation failures when total free memory existed but no contiguous block was large enough. The comparison with Best-Fits provides the most valuable metrics. It showed that First-Fit usually required fewer hole inspections, which means it made decisions faster and with less search work. However, Best-Fit performed better in the balanced mixed-size scenario because it preserved a larger block for a later large request and therefore allocated all four processes successfully. This shows that First-Fit is best when simplicity and speed matter most, while Best-Fit is best when memory is tight, process sizes vary significantly, and allocation success is more important for process future memory requests than search overhead.

References

- Bazuku, R., Anab, A., Gyemerah, S., & Daabo, M. I. (2023). An overview of computer operating systems and emerging trends. *Asian Journal of Research in Computer Science*, 16(4), 161-177. <https://doi.org/10.9734/AJRCOS/2023/v16i4380>
- CSU Global. (n.d.). *Lecture 4: Memory management* [Interactive lecture]. CSC507: Foundation of Operating Systems. Colorado State University Global.
- Esmacili, M., & Toghraee, M. (2024). Understanding concurrent processing: Challenges in operating systems. *Journal of Computer Based Parallel Programming*, 9(3), 18-23.
- Pandikumar, S., Jakaraddi, H. R., & Sevugapandi, N. (2025). Chapter 7: Impact of AI in the design of operating systems: An overview. In N. Revathy & S. Pandikumar (Eds.), *Shaping the digital future: From algorithms to intelligence (pp. 64-71)*. QTanalytics® India.
- Wilson, P. R., Johnstone, M. S., Neely, M., & Boles, D. (1995). Dynamic storage allocation: A survey and critical review. *Lecture Notes in Computer Science*, 1-116. https://doi.org/10.1007/3-540-60368-9_1